
ETA: Learning Implicit User Preference via Editing Trajectory Anchoring

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Language models are typically aligned to population-level preferences, but person-
2 alization often relies on only a handful of user demonstrations. Existing methods
3 learn from finalized outputs or explicit pairwise preferences, discarding the edit
4 trajectories that produced them. We observe that these trajectories provide process-
5 level supervision: they reveal which spans were rewritten, which alternatives were
6 rejected, and which edits remained stable through to the final draft. To exploit this
7 signal, we introduce Editing Trajectory Anchoring (ETA), a general framework for
8 converting user edit histories into preference supervision. ETA mines cleaned edit
9 logs for meaningful edit events and transforms each into a preference pair anchored
10 to the user’s polished final output, optimized with a standard preference objec-
11 tive such as DPO. By learning from the editing process rather than the outcome
12 alone, ETA turns each user revision into multiple informative training signals while
13 using edit logs only at training time. We evaluate ETA’s ability to learn personal-
14 ized alignment on real-writer benchmarks covering creative, argumentative, and
15 academic writing. Across these settings, ETA consistently outperforms few-shot
16 prompting, supervised fine-tuning, and other demonstration-based methods in win
17 rate. These results show that ETA turns revision trajectories into a denser and
18 more effective supervision signal than finalized outputs alone, without requiring
19 additional pairwise preference annotation beyond the user’s natural edit history.

20 1 Introduction

21 Large language models are typically aligned for general-purpose use, using feedback aggregated
22 across many annotators to reflect broad population-level preferences (Ouyang et al., 2022; Rafailov
23 et al., 2023). In practice, however, these models are often deployed in settings where the desired
24 behavior is far more specific: a model must adapt to the goals, constraints, style, and revision norms
25 of a particular user or task. This creates a mismatch between population-level alignment and local
26 preferences. Outputs that are broadly acceptable may still be misaligned for a specific user, especially
27 in open-ended generation tasks where acceptable responses depend on context that is difficult to
28 fully specify in a prompt (Santurkar et al., 2023; Chakrabarty et al., 2024). The central challenge
29 is therefore not only how to align models, but how to do so from the small amount of user-specific
30 evidence that is realistically available.

31 Existing alignment methods provide only partial solutions to this data bottleneck. RLHF and
32 preference-optimization methods such as DPO align language models effectively, but their strongest
33 results rely on large collections of human feedback or preference comparisons that are impractical to
34 obtain per-user (Ouyang et al., 2022; Rafailov et al., 2023). Demonstration-based methods reduce
35 this burden by learning from a small number of user-approved outputs (Shaikh et al., 2025). DITTO,
36 for instance, scaffolds fewer than ten polished demonstrations into a preference dataset by treating
37 the user’s outputs as preferred over the model’s own samples. Yet the proposals the user rejected,

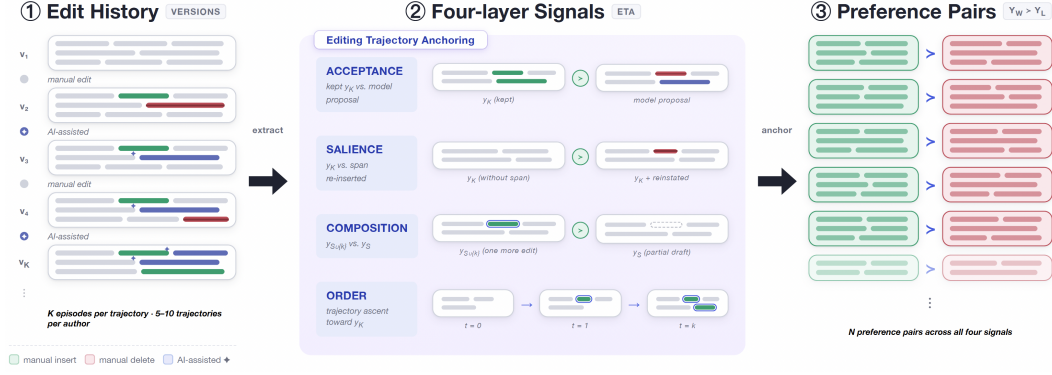


Figure 1: **ETA mines four distinct preference signals from each editing trajectory, each with its own natural anchor.** By learning from the editing process rather than the outcome alone, ETA turns each user revision into multiple informative training signals while using edit logs only at training time.

the spans they deleted, and the intermediate drafts they advanced past never enter the loss. Recent work has begun to mine edits themselves, treating them as supervised re-alignment targets (Liu et al., 2022) or distilling them into a latent preference description (Gao et al., 2024). These methods surface part of the editing process, but compress the trajectory into a single representation rather than reading it as multiple distinct preference signals. We ask whether the revision process can be converted directly into training signal, without being discarded or compressed into a single artifact. This is challenging because edits are heterogeneous in intent (Yang et al., 2017; Du et al., 2022; Jiang et al., 2022). Naively contrasting every intermediate state against the final draft conflates copy-editing and structural revisions with substantive preference-bearing changes, producing noisy and contradictory pairs. Our insight is that an editing trajectory contains several distinct preference signals, each admitting its own natural comparison structure.

We introduce *Editing Trajectory Anchoring* (ETA), a framework that converts a user’s edit history into preference supervision without soliciting any new annotations (see Fig. 1). Rather than treating preference learning as an act of collection in which the user is asked to compare candidates, we show that the act of revision itself produces dense preference signal, since each revision step is by construction a comparison the user has already made. Concretely, ETA mines four kinds of signal off each trajectory and uses each to build preference comparisons natively. *Acceptance* treats the polished final draft as preferred over a model proposal the user rejected or kept editing past. *Salience* treats the polished draft as preferred over a counterfactual in which a deleted span has been re-inserted. *Composition* treats a partial draft state with one additional accepted edit as preferred over the same state without it on the lattice of accepted edits. *Order* is captured implicitly by the lattice direction, since composition advances along an author-consistent path toward the polished draft. ETA mines each signal in its native form, producing many preference comparisons from one trajectory, and combines them under a standard preference objective such as DPO (Rafailov et al., 2023). This view reframes personalized alignment as a problem of *recovering* preferences the user has already revealed rather than collecting new ones, since the choices a writer makes while revising are precisely what sets them apart from anyone else and conventional fine-tuning on the finalized draft silently discards them. ETA is in essence a way of reading the trajectory back in (§3).

We evaluate ETA on two real-writer benchmarks, built on the CoAuthor (Lee et al., 2022) and ScholaWrite (Le et al., 2025) datasets, covering creative, argumentative, and academic writing. Across both settings, ETA consistently outperforms zero/few-shot prompting, SFT, and other demonstration-based alignment methods (Shaikh et al., 2025). These results indicate that revision histories carry preference structure that finalized outputs alone cannot supply, and that recovering this structure, rather than soliciting fresh comparisons, is a viable route to data-efficient personalization.

2 Related Work

Preference Finetuning and alignment for LLMs. Modern alignment of large language models trains on a learned preference signal. RLHF reduces human pairwise judgments to a reward model

that guides policy updates (Ouyang et al., 2022), and direct preference optimization reformulates the same pairwise objective as a closed-form likelihood ratio that no longer requires an explicit reward model (Rafailov et al., 2023). A growing family of variants generalizes the offline preference loss (Azar et al., 2024; Ethayarajh et al., 2024; Meng et al., 2024), replaces PPO with a group-relative on-policy alternative (Shao et al., 2024), or extends pairwise comparisons to listwise rankings (Liu et al., 2025). Each of these methods assumes access to a sizable preference corpus on the order of thousands to hundreds of thousands of comparisons, and even data-efficient regimes argue for a smaller but still nontrivial budget (Zhou et al., 2023). ETA reduces this requirement further by recovering preference pairs directly from a single user’s recorded edit trajectories, with no additional pairwise annotation.

Aligning LLMs from User Demonstrations. A complementary line aligns models to a specific user from a handful of polished examples rather than a large preference corpus. DITTO scaffolds fewer than ten user-supplied demonstrations into a preference dataset by treating each demonstration as preferred over the model’s own outputs and earlier checkpoints (Shaikh et al., 2025), group preference optimization extends this idea to cluster-level alignment from sparse few-shot evidence (Zhao et al., 2024), and related work personalizes outputs from prior user opinions (Hwang et al., 2023) or from prompt-based demonstrations elicited at inference (Brown et al., 2020; Zamfirescu-Pereira et al., 2023). Personalization benchmarks such as LaMP have begun to standardize this evaluation (Salemi et al., 2024), and recent variants of preference optimization use a base-model anchor to mitigate forgetting during personalized fine-tuning (Lee et al., 2024). Each method in this line treats the user’s polished output as the entire authorial signal, leaving the path that produced that output unobserved. ETA restores that path: the rejected proposals, the deleted spans, and the intermediate draft states inside a single trajectory each become preference pairs in their own right, multiplying the supervision per demonstration without any new annotation.

Aligning LLMs from Edit Histories. Edits and revisions have long been studied as a data source, but largely outside the preference-learning frame. Earlier work mines revision corpora for edit-intent classification (Yang et al., 2017), atomic-edit modeling (Faruqui et al., 2018), command-conditioned text editing (Faltings et al., 2021), document-level revision reasoning (Spangher et al., 2022), and iterative writing-process modeling (Du et al., 2022; Jiang et al., 2022), treating edits as supervised generation or classification targets rather than as comparisons. A more recent line does convert edits into alignment signal: Second Thoughts uses a chain-of-edits to re-align models with human values (Liu et al., 2022), while PRELUDE and Latent Preference Coding compress edit traces into a latent preference description that conditions inference (Gao et al., 2024). None of these methods extract more than one preference signal from a trajectory: edits enter their loss as a single supervised target or are compressed into a single latent code. ETA closes this gap by mining four distinct signals from every trajectory, each at its own natural anchor.

3 ETA: Editing Trajectory Anchoring

While prior work uses thousands of comparisons to align LLMs to a population, we instead align to a single user from a small set of recorded edit trajectories. Each trajectory carries multiple distinct preference signals, each with its own natural anchor. ETA mines them offline, constructs anchored preference pairs from each, and combines them under a standard preference objective such as DPO (Rafailov et al., 2023).

3.1 Setup and Notation

A language model can be viewed as a policy $\pi_\theta(y \mid x)$ that produces a distribution over completions y to a prompt x . We assume access to a base instruction-tuned reference policy π_{ref} . Standard preference-based fine-tuning trains π_θ to maximize a preference-derived reward subject to a KL constraint to the reference policy (Ziegler et al., 2019; Ouyang et al., 2022):

$$\max_{\pi_\theta} \mathbb{E}_{x, y \sim \pi_\theta(\cdot \mid x)} [r(x, y)] - \beta \mathbb{E}_x [D_{\text{KL}}(\pi_\theta(\cdot \mid x) \parallel \pi_{\text{ref}}(\cdot \mid x))]. \quad (1)$$

DPO (Rafailov et al., 2023) reparameterizes Eq. 1 into a closed-form pairwise loss using the implicit reward $r_\theta(x, y) = \log(\pi_\theta(y \mid x) / \pi_{\text{ref}}(y \mid x))$, eliminating the need for an explicit reward model. For each user, we further assume access to a set of recorded *edit trajectories*. A trajectory $T = [e_1, \dots, e_K]$ is an ordered sequence of *episodes*. Each episode e_k records a state before k immediately

Table 1: Preference structures ETA reads off each editing trajectory. Three of the four are mined as explicit preference pairs at their respective natural anchors. The fourth is captured implicitly through composition’s replay order. Sub-types differ in how the construction handles ambiguous cases (e.g. fuzzy-context salience reconstruction, manually-anchored composition).

Signal	Native anchor (chosen $\succeq$ rejected)	Sub-types
Acceptance	$y_K \succeq$ model proposal	rejection vs. override
Salience	$y_K \succeq y_K$ with deleted span re-inserted	strict vs. fuzzy reconstruction
Composition	$y_{S \cup \{k\}} \succeq y_S$ for $k \in A \setminus S$	full vs. manually-anchored ascent
Order	captured via composition replay order	no pairs emitted

before the edit, the state after _{k} immediately after, a decision label $d_k \in \{\text{accepted, rejected, } \emptyset\}$ when the editing UI surfaces a model suggestion, and the model proposal p_k that triggered the episode (when present). The *final draft* y_K is the polished text the author shipped at the end of the trajectory. The set of accepted-edit indices $A \subseteq \{1, \dots, K\}$ induces a lattice 2^A of partial *replay states* y_S for any subset $S \subseteq A$, with $y_\emptyset$ the initial document state and $y_A = y_K$. Throughout this section, the prompt x is the task instruction shared across the trajectory, model proposals p_k are the bare span-level continuations the user saw at edit time, and partial replay states y_S are produced by applying the edits in S in their original temporal order via a deterministic textual splice that does not assume edit operations commute. The personalization task is to align π_{ref} to a per-author policy π_θ from T alone.

3.2 Editing Trajectory Anchoring

An editing trajectory carries information about the writing process that the finalized text discards. The final draft tells us only what the user wanted in the end, while the trajectory tells us what they noticed and changed along the way, including the spans they re-selected, the clauses they rewrote, and the model suggestions they manually overrode. Each of these revisions doubles as a source of negatives. Rejected proposals, overwritten phrases, and manual corrections are states the user actively chose against, in contrast to synthetic on-policy samples whose relationship to the user’s preferences is indirect. Contrastive learning theory motivates such revealed-preference negatives as a source of stronger gradient signal per pair (Robinson et al., 2021). The trajectory also surfaces which parts of the draft were authorially important. Repeated edits, large localized rewrites, and active deletions all mark spans whose final form mattered enough to revise, information that vanishes once a single finalized text is committed. Beyond these endpoint signals, a trajectory captures editing policy as well, for instance whether the user softens tone before adding specificity, or prefers manual edits for content but model help for wording. ETA mines the first three of these into preference pairs through the acceptance, salience, and composition signals introduced below. The fourth, procedural preference, surfaces only implicitly through the lattice-ascent direction in composition.

Existing demonstration-only methods anchor every preference pair at the polished final draft y_K and synthesize the rejected side from on-policy samples (Shaikh et al., 2025), compressing the trajectory into a single signal at a single anchor. ETA instead mines pairs from three explicit signals (acceptance, salience, composition), each at its own natural anchor. The temporal-order signal is captured implicitly through the replay direction of the composition pairs. Table 1 summarizes the resulting structure, and Algorithm 1 states the offline pair-construction loop.

Acceptance. Whenever the editing UI surfaces a model suggestion, the user makes a binary decision. The *rejection* sub-type is the strongest version of the signal: the user saw the proposal and turned it down, so the native pair is $y_K \succeq p_k$. The *override* sub-type is informative even when the user clicked accept: they kept editing afterwards, so y_K differs from the proposal as shipped. The native pair is again $y_K \succeq p_k$, where the difference between the proposal and y_K captures what the user later removed or replaced.

Salience. Spans the user actively deleted reveal what they did not want in the final draft. The native comparison is between y_K and a reconstructed text that re-inserts the deleted span at its original position. Two reconstruction tiers stratify the same salience signal by reconstruction confidence: a *strict* tier requires the surrounding context to survive intact in y_K , while a *fuzzy* tier falls back to a

Algorithm 1 ETA offline pair construction.

Require: trajectory T , polished final draft y_K , accepted-edit set A

```
1: for each rejected episode  $k$  do
2:   emit ( $y_K, p_k$ )                                // acceptance / rejection sub-type
3: end for
4: for each accepted episode  $k$  with  $p_k \neq y_K$  do
5:   emit ( $y_K, p_k$ )                                // acceptance / override sub-type
6: end for
7: for each deleted span identified by diff of before $k$  and after $k$  do
8:    $\tilde{y}_K \leftarrow y_K$  with the span re-inserted at its original position
9:   emit ( $y_K, \tilde{y}_K$ )                                // salience
10: end for
11: for  $N_D$  samples per trajectory do
12:   sample  $S \subseteq A$  uniformly at random and  $k \in A \setminus S$  uniformly at random
13:   emit ( $y_{S \cup \{k\}}, y_S$ )                        // composition (lattice ascent)
14: end for
```

168 Levenshtein-similar match snapped to a word boundary. The full extractor, including seven quality
169 filters that strip whitespace-only spans, typo corrections, scaffolding phrases, and autocomplete
170 repetition, is described in the appendix.

171 **Composition.** The set of accepted edits induces a partial order on draft states: y_S precedes $y_{S \cup \{k\}}$
172 for any S missing edit $k \in A$. The native composition preference is one-step-more-edits $\succeq$ one-step-
173 fewer-edits, with both sides drawn from the same author’s accepted edits in a consistent order. This is
174 the only signal whose chosen side is a partial replay state rather than the polished final draft, so the
175 comparison anchors on lattice position rather than on y_K . A *manually-anchored* variant restricts k to
176 lattice steps that include a manual-typed (rather than model-accepted) chunk, isolating the slice of
177 composition with the highest authorial specificity.

178 **Order.** The temporal ordering of edits is observable in every trajectory, and ETA treats it implicitly
179 through the composition signal’s lattice-ascent direction. We deliberately use temporal *order* as the
180 only time-derived signal, not wall-clock duration or other proxies for editing effort, because effort
181 does not cleanly map to preference strength. A long pause may signal writer’s block rather than
182 strong preference, and a short pause may follow an obvious choice.

183 Three properties follow from this construction. All pairs are offline, with pair count bounded by
184 trajectory content rather than the trainer’s sampling budget. Each negative is one the user actively
185 chose against, providing comparisons whose per-pair informativeness we test empirically (§5). Each
186 pair belongs to exactly one signal, making per-signal weighting the natural lever in the loss.

187 3.3 Importance-Weighted DPO Objective

188 Each preference pair carries a per-signal weight w_i determined by the signal that produced it. The
189 training objective is a per-pair-weighted DPO loss with mean-1 normalization on the weights:

$$\mathcal{L}_{ETA}(\theta) = -\mathbb{E}_{(x, y^+, y^-, \tilde{w}) \sim \mathcal{D}} [\tilde{w} \cdot \log \sigma(\beta(r_\theta(x, y^+) - r_\theta(x, y^-)))], \quad (2)$$

190 where r_θ is the implicit reward defined in §3.1, σ is the logistic function, β is the DPO temperature,
191 and $\mathcal{D}$ is the union of all signal-induced pairs from §3.2. Per-pair weights are normalized as
192 $\tilde{w}_i = w_i / \text{mean}_j(w_j)$ so the dataset mean is exactly 1, which keeps the effective loss scale identical
193 to standard DPO and lets us tune β and the learning rate independently of the chosen weight tuple.
194 The construction is agnostic to the choice of preference loss: replacing the Bradley-Terry term in
195 Eq. 2 with KTO’s prospect-theoretic objective (Ethayarajh et al., 2024) or SimPO’s reference-free
196 margin (Meng et al., 2024) yields variants of ETA that retain the per-signal weighting without
197 modification. Optimizer hyperparameters and the production per-signal weight tuple are reported in
198 the appendix.

3.4 Theoretical Validity of Per-Signal Anchoring

A natural concern about Eq. 2 is whether mixing pairs with heterogeneous anchors (Table 1) is well-defined under DPO.

Proposition 1 (Anchor-agnostic preference combination). *Let $\mathcal{D} = \bigcup_{m=1}^M \mathcal{D}_m$ be the union of preference-pair datasets arising from ETA’s M signals, where each $\mathcal{D}_m$ uses its own anchor convention. The loss in Eq. 2 is well-defined on $\mathcal{D}$, factorizes additively over pairs, and admits an optimal policy π_θ^* whose implicit reward $r_\theta^*(x, y) = \log(\pi_\theta^*(y | x) / \pi_{\text{ref}}(y | x))$ is consistent with the union of all signal-induced preferences whenever those preferences are jointly compatible with some underlying user reward.*

Proof sketch. The DPO reparameterization (Rafailov et al., 2023) maps any reward r to a KL-constrained optimal policy via $r(x, y) = \beta \log(\pi^*(y | x) / \pi_{\text{ref}}(y | x)) + \beta \log Z(x)$, well-defined for any pair (x, y) . Each Bradley-Terry term in Eq. 2 depends only on the difference $r_\theta(x, y^+) - r_\theta(x, y^-)$, so $Z(x)$ cancels and heterogeneous anchors contribute independent additive terms. Rafailov et al. (2024) extend this reparameterization to the token-level MDP, confirming that the Bradley-Terry constraint is inherently pairwise. The optimum is therefore the policy whose implicit reward is consistent with the union of signal-induced preferences. $\square$

Proposition 1 is a consistency result. It shows mixing heterogeneous anchors is well-defined, but does not establish per-signal validity or non-redundancy. Those are empirical questions, justified by §3.2’s design rationale and the per-signal ablation in §5 respectively.

4 Experiments

We outline the data, preprocessing, baselines, and evaluation protocol below, then report results in §5.

Data. Measuring few-shot alignment with ETA requires editing trajectories from individuals, not aggregated text corpora. We therefore build on two real-writer datasets that record per-user edit events at episode-level granularity. CoAuthor (Lee et al., 2022) is a Stanford collaborative-writing corpus that logs mixed-genre creative and argumentative writing sessions as Quill Delta event streams, with explicit GPT-3 model proposals and accept-or-reject decisions logged at every suggestion. ScholaWrite (Le et al., 2025) is an academic-writing corpus of Overleaf paragraph-level keystroke trajectories from graduate-student authors. CoAuthor’s recorded decision labels and model proposals populate ETA’s acceptance signal directly. ScholaWrite contains no model proposals in the trace, so the acceptance signal is structurally empty there and ETA trains on the salience and composition signals alone, a stress test for whether the framework gracefully handles benchmarks where some signals do not fire. Per-episode field schemas and example trajectories are in Appendix A.

Preprocessing. The two datasets differ in scale: CoAuthor’s collaborative-writing logs span dozens of authors, while ScholaWrite’s logs come from LaTeX projects. To match the per-benchmark scale of prior demonstration-based personalization work (Shaikh et al., 2025), we randomly select ten CoAuthor authors from candidates with at least twelve valid sessions across both genres, yielding six creative and six argumentative trajectories per author. ScholaWrite’s seven authors are all graduate-student contributors to the project who satisfy a minimum event-number threshold. We divide the data into an 8:2:2 split per author on both datasets, capping at twelve trajectories per author. The per-trajectory fields populated for both benchmarks $(x, y_K, \text{episode chain } [e_1, \dots, e_K])$ match the notation introduced in §3.1. Episode derivation, scaffold-stripping filters, and trajectory-level field schemas are in Appendix A.2.

Baselines. Following prior work on demonstration-based personalization (Shaikh et al., 2025), we use Mistral-7B-Instruct-v0.2 (Jiang et al., 2023) as a starting point and train using LoRA (Hu et al., 2022) adapters. All trained methods share the same supervised fine-tuning recipe before any preference-tuning phase, so any differences across methods isolate the effect of the preference-data construction. We compare ETA against five baselines. **Zero-shot Mistral** runs the base model on the task prompt with no demonstrations. **Few-shot Mistral** adds the entire train set of an author’s demonstrations in-context. **SFT** fine-tunes a LoRA adapter on the author’s polished final drafts y_K alone. **DITTO** (Shaikh et al., 2025) is the closest prior work to ETA: it also aligns from a

small number of per-user demonstrations and recovers preference pairs from those demonstrations rather than soliciting new annotations. DITTO performs SFT followed by DPO with y_K as the chosen side and online-sampled negatives generated by the model itself during training. The key difference from ETA is that DITTO synthesizes its rejected pool through online sampling and does not consume the user’s recorded edit history. **DITTO-step** is a step-level baseline derived in this work. Because the original DITTO does not consume edit-history information, we adapt it to do so, which lets any gap with ETA isolate the multi-anchor design from the benefit of using edit history at all. DITTO-step augments DITTO with per-episode offline preference pairs whose chosen side is y_K and whose rejected side is reconstructed from each episode’s recorded edit signal. These offline pairs sit alongside DITTO’s online resampling cache rather than replace it. Detailed DITTO-step pair construction, full optimizer hyperparameters, and the production per-signal weight tuple are in Appendix B.

Evaluation. The two benchmarks contain 17 authors total (10 CoAuthor and 7 ScholaWrite). Evaluating six methods on each yields 102 author-method conditions (60 on CoAuthor and 42 on ScholaWrite). To facilitate evaluation at this scale, we use **GPT-4o** as an LLM judge to compare method outputs head-to-head. Prior work has used LLMs to both annotate and evaluate text generations (Zheng et al., 2023), and model-based classification has been shown to be more reliable than non-expert humans for authorship and style similarity (Krishna et al., 2020; Hallinan et al., 2023; Liu et al., 2024; Liu and May, 2024). We follow the head-to-head authorship-style protocol of Shaikh et al. (2025): given an author’s reference text t and two generations a, b from different methods, the judge selects which generation more closely matches t ’s style. To control for ordering bias, every comparison is judged twice with the order of a and b swapped. We report per-method head-to-head win rates aggregated across authors and prompts, alongside one-way ANOVA across methods and Tukey HSD pairwise comparisons at $\alpha = 0.05$. Sampling configuration, scoring conventions, and the per-author noise floor are in Appendix B.2. To mitigate judge bias, all comparisons use position-swap averaging; results are consistent with ANOVA across authors rather than relying on any single pairwise call.

5 Results

Our main results, evaluated with GPT-4o, are summarized in Table 2. Averaged across both benchmarks, ETA outperforms all baselines: 65.00% on CoAuthor and 61.43% on ScholaWrite. The strongest baseline differs by benchmark, with DITTO-step at 59.17% on CoAuthor and SFT at 55.71% on ScholaWrite. On CoAuthor, ETA achieves the per-cell column maximum on five of ten authors (a_2, a_3, a_4, a_6, a_7), DITTO-step on three (a_1, a_5, a_8), and few-shot Mistral and SFT on one each (a_9 and a_{10}). On ScholaWrite, the per-author plurality goes to SFT with three of seven authors (a_1, a_3, a_7), ETA wins two (a_2, a_5), DITTO-step wins a_4 , and few-shot Mistral wins a_6 . SFT serves as a strong baseline on both benchmarks (44.00% on CoAuthor, 55.71% on ScholaWrite). ETA provides a +21.00 pp average improvement over SFT on CoAuthor and a +5.72 pp improvement on ScholaWrite.

Prompted baselines lag far behind ETA, especially zero-shot Mistral (−30.83 pp on CoAuthor and −25.48 pp on ScholaWrite). Adding all training-set demonstrations as a few-shot prompt helps substantially: few-shot Mistral lifts +20.16 pp on CoAuthor and +13.10 pp on ScholaWrite over zero-shot. The lift is larger on CoAuthor in absolute terms but few-shot becomes relatively more competitive on ScholaWrite, where it climbs above DITTO-step (+2.15 pp) into third place among the six methods. Even so, few-shot prompting still falls −10.67 pp behind ETA on CoAuthor and −12.38 pp on ScholaWrite, suggesting that the underlying instruction-following priors of an out-of-the-box LM cannot be re-shaped by prompting alone for a specific user’s voice.

Among trained baselines, the picture differs across benchmarks. On CoAuthor, the closest competitor to ETA is DITTO-step (59.17%), the trajectory-aware variant we derived to give DITTO access to edit-history information, with a gap of 5.83 pp to ETA. DITTO and SFT cluster more than 20 pp behind at 43.33% and 44.00%, indicating that consuming the recorded edit history is itself a meaningful lift over methods that learn only from the polished final draft. On ScholaWrite, the picture inverts: SFT is the strongest baseline (55.71%) and DITTO-step drops to 46.90%, the lowest of any trained method. ScholaWrite contains no recorded model proposals (§4), so DITTO-step’s signature draft-prefix-plus-proposal offline pair degenerates to a snapshot-based variant where the rejected

Data	Method	a_{avg}	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	a_{10}
CoAuthor	zero-shot	34.17 _{4.86}	18.33	33.33	13.33	50.00	51.67	60.00	33.33	21.67	26.67	33.33
	few-shot	54.33 _{2.63}	58.33	40.00	53.33	56.67	41.67	55.00	56.67	60.00	68.33	53.33
	SFT	44.00 _{3.87}	40.00	61.67	36.67	41.67	43.33	26.67	55.00	40.00	31.67	63.33
	DITTO	43.33 _{3.79}	43.33	30.00	61.67	48.33	38.33	21.67	36.67	51.67	55.00	46.67
	DITTO-step	59.17 _{3.67}	83.33	50.00	61.67	45.00	65.00	65.00	48.33	65.00	60.00	48.33
	ETA (ours)	65.00 _{3.04}	56.67	85.00	73.33	58.33	60.00	71.67	70.00	61.67	58.33	55.00
ScholaWrite	zero-shot	35.95 _{3.72}	30.00	40.00	28.33	21.67	36.67	48.33	46.67	-	-	-
	few-shot	49.05 _{6.35}	20.00	40.00	50.00	58.33	65.00	68.33	41.67	-	-	-
	SFT	55.71 _{7.22}	71.67	46.67	83.33	33.33	56.67	33.33	65.00	-	-	-
	DITTO	50.95 _{4.05}	45.00	46.67	63.33	55.00	33.33	63.33	50.00	-	-	-
	DITTO-step	46.90 _{6.28}	66.67	50.00	26.67	66.67	31.67	33.33	53.33	-	-	-
	ETA (ours)	61.43 _{5.05}	66.67	76.67	48.33	65.00	76.67	53.33	43.33	-	-	-

Table 2: **GPT-4o Eval:** Head-to-head win rates (%) between methods on real-writer benchmarks built on the CoAuthor (Lee et al., 2022) and ScholaWrite (Le et al., 2025) datasets. ETA outperforms all baseline methods on average across both benchmarks. a_i denotes a model trained on the editing trajectory of author i . Numbers are pairwise win rates with position-swap averaging. The average column reports mean and standard error of the mean across authors. Each block is evaluated independently under the same six-method, GPT-4o judged protocol. CoAuthor (10 authors) and ScholaWrite (seven authors a_1 – a_7) were evaluated under the same protocol. The trailing three columns of the ScholaWrite rows are marked – because no a_8 – a_{10} exists in that benchmark.

Table 3: **Source-component ablation on CoAuthor.** Round-robin head-to-head win rates (%) within a six-method cohort consisting of full ETA and the five single-signal drop variants, GPT-4o judge. Every signal removed moves average win rate downward. One-way ANOVA across the six cohort members returns $F(5, 54) = 0.59$, $p > 0.05$. Tukey HSD finds no individual drop significant at $\alpha = 0.05$, consistent with a per-author standard error of about 5 pp at $n = 10$. The directional unanimity is the claim, not per-drop significance.

Variant	a_{avg} (%)	Δ vs full
ETA (full)	53.50	n/a
→ remove rejection	49.67	−3.83
→ remove override	47.67	−5.83
→ remove salience	46.33	−7.17
→ remove full-lattice composition	49.50	−4.00
→ remove manually-anchored composition	53.33	−0.17

side is an earlier full-document draft. This snapshot variant carries weaker contrastive signal than the proposal-based variant on CoAuthor, and DITTO-step underperforms DITTO on ScholaWrite (46.90% vs 50.95%), the only setting in our cohort where adding edit-history-based offline pairs hurts. ETA is the most stable method across the two benchmarks: a −3.57 pp drop from CoAuthor to ScholaWrite versus DITTO-step’s −12.26 pp drop, the largest swing in the cohort. While the author counts reflect the scale of available real-writer trajectory datasets, the per-author gains are consistent across both benchmark surfaces, suggesting the pattern is not an artifact of individual authors.

One-way ANOVA across the six methods gives $F(5, 54) = 9.57$, $p < 0.001$ on CoAuthor and $F(5, 36) = 2.38$, $p = 0.06$ on ScholaWrite, where the smaller author count limits power. Tukey HSD pairwise comparisons at $\alpha = 0.05$ confirm ETA’s lead over zero-shot, SFT, and DITTO on CoAuthor and over zero-shot on ScholaWrite. Other leads are directional but inside the Tukey non-significance band, consistent with a per-author standard error of about 5 pp at $n = 10$ and larger noise at $n = 7$.

5.1 Analysis

Source-Component Ablation. To test whether each preference signal contributes information the others cannot recover, we re-train ETA five times on CoAuthor, each time removing one signal from the pair construction while holding all production hyperparameters fixed. We ablate on CoAuthor because all four signal types fire there; ScholaWrite’s structurally absent acceptance signal would conflate signal removal with recording-surface limitation. The five drop variants ablate the two acceptance sub-signals (rejection and override), the salience signal, and the two composition variants (full-lattice ascent and manually-anchored ascent). Each variant is evaluated head-to-head against

the full ETA and the four other drop variants in a six-method round-robin cohort, under the same protocol as §4.

Every signal removal moves the average win rate downward (Table 3). The salience signal contributes the largest amount of unique information (-7.17 pp when removed), followed by the override sub-signal of acceptance (-5.83), the full-lattice variant of composition (-4.00), the rejection sub-signal of acceptance (-3.83), and the manually-anchored variant of composition (-0.17). The per-author standard error of about 5 pp at $n = 10$ rules out Tukey-significance for individual drops at this scale, so the claim of this experiment is the directional unanimity rather than per-drop significance: while no individual signal removal reaches Tukey significance at $n = 10$, the unanimous directional pattern (all drops negative) provides evidence against redundancy.

Per-Author Patterns. Aggregate win rates hide author-level structure. To probe whether per-author crowns are random or grounded in data, we computed eleven trajectory-level metrics per author and ran Spearman correlations against each method’s per-author win rate, separately on each benchmark (details in Appendix B.5). The crowns map cleanly onto trajectory characteristics, and each method has a distinct winning surface. DITTO-step wins where the recording surface gives it dense, sharp signal: a_1 ’s 83.33% column-max on CoAuthor (25 pp margin) coincides with that author’s 94% proposal-acceptance rate and lowest type-token-ratio stdev, and across the ten CoAuthor authors DITTO-step’s per-author win rate tracks low type-token-ratio stdev at Spearman $\rho = -0.71$. ETA wins where trajectories are signal-rich: its column-maxes on CoAuthor cluster on authors with longer final drafts ($\rho = +0.59$) and higher draft-length variance ($\rho = +0.56$), where multi-anchor mining benefits from richer revisions. SFT wins where the polished draft itself becomes the dominant supervisory signal: a_3 on ScholaWrite, the largest per-author cell across both benchmarks, combines the cohort’s longest draft (1119 characters) with only 15 trajectory pairs ETA can mine, and SFT exploits the rich polished target without competition. DITTO has no winning surface on CoAuthor and is never a column-max, because its on-policy negative sampling does not latch onto any per-trajectory feature.

6 Conclusion

Current alignment recipes require many new annotations to specialize a model to a single user. We showed that the comparisons needed for personalization are signals the user has already revealed while writing. We introduced ETA (Editing Trajectory Anchoring), a framework that mines preference pairs from recorded editing trajectories and combines them under a standard preference objective. Across two real-writer benchmarks covering creative, argumentative, and academic writing, ETA outperforms all baselines we test.

Limitations. ETA requires per-user editing trajectories, not just finalized demonstrations, and many applications do not capture this kind of trace. Even when they do, missing channels bound the achievable signal density per author. Our evaluation also uses a single base model family (Mistral-7B-Instruct-v0.2) and a single LLM judge (GPT-4o). We also do not control for total preference pair count across methods; pair-count effects are partially addressed by the DITTO-step comparison, which also uses offline pairs but underperforms ETA. Last but not least, the per-author sample sizes ($n = 10$, $n = 7$) limit statistical power; Tukey non-significance on several directional leads should be interpreted accordingly.

Future Work. Our analyses suggest ETA’s multi-anchor mining is more robust to recording-surface variation than single-mechanism baselines (Appendix B.6), bounded only by the signal density an author’s trajectory exposes (Appendix B.5). Two avenues follow. One deepens the robustness by recovering what the four paragraph-level signals here cannot reach. Document-level revisions (paragraph reordering, structural rewrites) and the procedural order of edits carry editorial intent that the trajectory exposes but ETA treats only implicitly, and a *recording-surface-aware* pair-construction policy could route them into the loss alongside the existing channels. A second raises the bound by treating the editing tool itself as part of the alignment pipeline. Today’s editing applications were built for productivity, collaboration, and version control, not for trajectory-aware alignment, and what they record bounds what ETA can mine. Co-designing editing applications and pair-construction policies, exposing proposals, deletion spans, and accepted-edit lattices as first-class outputs, turns recording-surface adequacy into a guarantee rather than a discovery.

References

- Mohammad Gheshlaghi Azar, Zhaohan Daniel Guo, Bilal Piot, Remi Munos, Mark Rowland, Michal Valko, and Daniele Calandriello. 2024. A general theoretical paradigm to understand learning from human preferences. In *International Conference on Artificial Intelligence and Statistics*. PMLR, pages 4447–4455.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33:1877–1901.
- Tuhin Chakrabarty, Philippe Laban, Divyansh Agarwal, Smaranda Muresan, and Chien-Sheng Wu. 2024. Art or artifice? large language models and the false promise of creativity. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. pages 1–34.
- Wanyu Du, Vipul Raheja, Dhruv Kumar, Zae Myung Kim, Melissa Lopez, and Dongyeop Kang. 2022. Understanding iterative revision from human-written text. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. pages 3573–3590.
- Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. 2024. Model alignment as prospect theoretic optimization. In *Forty-first International Conference on Machine Learning*. <https://openreview.net/forum?id=iUwHnoENnl>.
- Felix Faltings, Michel Galley, Gerold Hintz, Chris Brockett, Chris Quirk, Jianfeng Gao, and William B Dolan. 2021. Text editing by command. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. pages 5259–5274.
- Manaal Faruqui, Ellie Pavlick, Ian Tenney, and Dipanjan Das. 2018. Wikiatomicedits: A multilingual corpus of wikipedia edits for modeling language and discourse. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. pages 305–315.
- Ge Gao, Alexey Taymanov, Eduardo Salinas, Paul Mineiro, and Dipendra Misra. 2024. Aligning llm agents by learning latent preference from user edits. *Advances in neural information processing systems* 37:136873–136896.
- Skyler Hallinan, Faeze Brahman, Ximing Lu, Jaehun Jung, Sean Welleck, and Yejin Choi. 2023. STEER: Unified style transfer with expert reinforcement. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, Singapore, pages 7546–7562. <https://doi.org/10.18653/v1/2023.findings-emnlp.506>.
- Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=nZeVKeeFYf9>.
- EunJeong Hwang, Bodhisattwa Majumder, and Niket Tandon. 2023. Aligning language models to user opinions. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. pages 5906–5919.
- Albert Qiaochu Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. 2023. Mistral 7b. *ArXiv abs/2310.06825*.
- Chao Jiang, Wei Xu, and Samuel Stevens. 2022. arxivedit: Understanding the human revision process in scientific writing. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. pages 9420–9435.
- Kalpesh Krishna, John Wieting, and Mohit Iyyer. 2020. Reformulating unsupervised style transfer as paraphrase generation. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu, editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Online, pages 737–762. <https://doi.org/10.18653/v1/2020.emnlp-main.55>.

427 Khanh Chi Le, Linghe Wang, Minhwa Lee, Ross Volkov, Luan Tuyen Chau, and Dongyeop
428 Kang. 2025. Scholawrite: A dataset of end-to-end scholarly writing process. *arXiv preprint*
429 *arXiv:2502.02904*.

430 Gihun Lee, Minchan Jeong, Yujin Kim, Hojung Jung, Jaehoon Oh, Sangmook Kim, and Se-Young
431 Yun. 2024. Bapo: Base-anchored preference optimization for overcoming forgetting in large
432 language models personalization. In *Findings of the Association for Computational Linguistics:*
433 *EMNLP 2024*. pages 6804–6820.

434 Mina Lee, Percy Liang, and Qian Yang. 2022. Coauthor: Designing a human-ai collaborative writing
435 dataset for exploring language model capabilities. In *Proceedings of the 2022 CHI conference on*
436 *human factors in computing systems*. pages 1–19.

437 Ruibo Liu, Chenyan Jia, Ge Zhang, Ziyu Zhuang, Tony Liu, and Soroush Vosoughi. 2022. Second
438 thoughts are best: Learning to re-align with human values from text edits. *Advances in Neural*
439 *Information Processing Systems* 35:181–196.

440 Shuai Liu, Shantanu Agarwal, and Jonathan May. 2024. Authorship style transfer with policy
441 optimization. *ArXiv abs/2403.08043*.

442 Shuai Liu and Jonathan May. 2024. Style transfer with multi-iteration preference optimization. In
443 *North American Chapter of the Association for Computational Linguistics*.

444 Tianqi Liu, Zhen Qin, Junru Wu, Jiaming Shen, Misha Khalman, Rishabh Joshi, Yao Zhao, Moham-
445 mad Saleh, Simon Baumgartner, Jialu Liu, et al. 2025. Lipo: Listwise preference optimization
446 through learning-to-rank. In *Proceedings of the 2025 Conference of the Nations of the Americas*
447 *Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume*
448 *1: Long Papers)*. pages 2404–2420.

449 Yu Meng, Mengzhou Xia, and Danqi Chen. 2024. Simpo: Simple preference optimization with a
450 reference-free reward. *Advances in Neural Information Processing Systems* 37:124198–124235.

451 Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong
452 Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to
453 follow instructions with human feedback. *Advances in neural information processing systems*
454 35:27730–27744.

455 Rafael Rafailov, Joey Hejna, Ryan Park, and Chelsea Finn. 2024. From $\$r\$$ to $\$q^{*\$}$:
456 Your language model is secretly a q-function. In *First Conference on Language Modeling*.
457 <https://openreview.net/forum?id=kEVcNxtqXk>.

458 Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea
459 Finn. 2023. Direct preference optimization: Your language model is secretly a reward model.
460 *Advances in neural information processing systems* 36:53728–53741.

461 Joshua David Robinson, Ching-Yao Chuang, Suvrit Sra, and Stefanie Jegelka. 2021. Contrastive
462 learning with hard negative samples. In *International Conference on Learning Representations*.
463 <https://openreview.net/forum?id=CR1XOQ0UTh->.

464 Alireza Salemi, Sheshera Mysore, Michael Bendersky, and Hamed Zamani. 2024. Lamp: When
465 large language models meet personalization. In *Proceedings of the 62nd Annual Meeting of the*
466 *Association for Computational Linguistics (Volume 1: Long Papers)*. pages 7370–7392.

467 Shibani Santurkar, Esin Durmus, Faisal Ladhak, Cinoo Lee, Percy Liang, and Tatsunori Hashimoto.
468 2023. Whose opinions do language models reflect? In *International conference on machine*
469 *learning*. PMLR, pages 29971–30004.

470 Omar Shaikh, Michelle S. Lam, Joey Hejna, Yijia Shao, Hyundong Justin Cho, Michael S.
471 Bernstein, and Diyi Yang. 2025. Aligning language models with demonstrated
472 feedback. In *The Thirteenth International Conference on Learning Representations*.
473 <https://openreview.net/forum?id=1qGkuxI9UX>.

- 474 Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang,
475 Mingchuan Zhang, YK Li, Yang Wu, et al. 2024. Deepseekmath: Pushing the limits of mathemati-
476 cal reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- 477 Alexander Spangher, Xiang Ren, Jonathan May, and Nanyun Peng. 2022. NewsEdits: A news
478 article revision dataset and a novel document-level reasoning challenge. In Marine Carpuat,
479 Marie-Catherine de Marneffe, and Ivan Vladimir Meza Ruiz, editors, *Proceedings of the 2022*
480 *Conference of the North American Chapter of the Association for Computational Linguistics:*
481 *Human Language Technologies*. Association for Computational Linguistics, Seattle, United States,
482 pages 127–157. <https://doi.org/10.18653/v1/2022.naacl-main.10>.
- 483 Diyi Yang, Aaron Halfaker, Robert Kraut, and Eduard Hovy. 2017. Identifying semantic edit
484 intentions from revisions in wikipedia. In *Proceedings of the 2017 Conference on Empirical*
485 *Methods in Natural Language Processing*. pages 2000–2010.
- 486 J Diego Zamfirescu-Pereira, Richmond Y Wong, Bjoern Hartmann, and Qian Yang. 2023. Why
487 johnny can’t prompt: how non-ai experts try (and fail) to design llm prompts. In *Proceedings of*
488 *the 2023 CHI conference on human factors in computing systems*. pages 1–21.
- 489 Siyan Zhao, John Dang, and Aditya Grover. 2024. Group preference optimization: Few-shot
490 alignment of large language models. In *The Twelfth International Conference on Learning*
491 *Representations*.
- 492 Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang,
493 Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E. Gonzalez, and Ion
494 Stoica. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Thirty-*
495 *seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
496 <https://openreview.net/forum?id=uccHPGDlao>.
- 497 Chunting Zhou, Pengfei Liu, Puxin Xu, Srinivasan Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia
498 Efrat, Ping Yu, Lili Yu, et al. 2023. Lima: Less is more for alignment. *Advances in Neural*
499 *Information Processing Systems* 36:55006–55021.
- 500 Daniel M Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B Brown, Alec Radford, Dario Amodei, Paul
501 Christiano, and Geoffrey Irving. 2019. Fine-tuning language models from human preferences.
502 *arXiv preprint arXiv:1909.08593*.

503 A Dataset and Benchmark Details

504 A.1 Task Example

505 Each editing trajectory $T = [e_1, \dots, e_K]$ pairs a task prompt x with a polished final draft y_K and an
506 ordered chain of episodes. Each episode carries a before-text before_k , an after-text after_k , a decision
507 label d_k (accepted, rejected, or empty for purely manual segments), and a model proposal p_k when
508 one was surfaced by the editing UI. We illustrate the schema with two examples trajectories on
509 creative and argumentative writing separately.

510 **Example 1: Creative.** *Prompt:* “When you’re 28, science discovers a drug that stops all effects of
511 aging... Write a story that starts with the following: People that have taken the drugs experience
512 loss of intelligence.” *Polished final draft* y_K : “People that have taken the drugs experience loss of
513 intelligence. When they are 40, their minds are destroyed. This creates a big problem to the society
514 because lots of people with low IQ wander in the streets...” Three representative episodes:

- 515 • e_1 d_1 rejected, $p_1 =$ “People that have taken the drugs and haven’t died, yet, will be locked
516 up in an institution for the rest of their lives.” (Author rejected this opening.)
- 517 • e_2 d_2 accepted, $p_2 =$ “When they reach 85, their bodies begin to fail.” Author accepted,
518 then later overrode in the polished draft to “When they are 40, their minds are destroyed.”
- 519 • e_5 d_5 accepted, $p_5 =$ “The winner will become immortal.” Accepted but its content does
520 not appear in y_K , contributing an override pair.

Table 4: **Dataset statistics.** Per-author trajectory counts after preprocessing. CoAuthor authors are a random sample of size 10 drawn from candidates with at least 12 valid sessions in both genres. ScholaWrite authors are the seven graduate-student contributors to a LaTeX project who satisfy a per-author cap of 12 trajectories.

Benchmark	Authors	Train/auth.	Val/auth.	Test/auth.	Total trajectories
CoAuthor	10	8	2	2	120
ScholaWrite	7	8	2	2	84

Example 2: Argumentative. *Prompt:* “Should Schools Provide Free Pads and Tampons?...”
Polished final draft y_K : “In my opinion, schools need to provide pads and tampons to their students as needed. I would suggest keeping a dispenser in each of the girl’s bathrooms and locker rooms...”
 Three representative episodes:

- $e_1 d_1$ accepted, p_1 = “The dispenser would be similar to those that dispense soap and paper towels, but it would contain pads and tampons.” Accepted and present nearly verbatim in y_K .
- $e_2 d_2$ rejected, p_2 = “Plus, if a student loses or forgets their tampon or pad that day, they would no longer have to worry about going on a scavenger...”
- $e_3 d_3$ rejected, p_3 = “I think a period dispenser would be a great idea...” Two consecutive rejections at the same draft prefix yield two acceptance-rejection pairs anchored at y_K .

On ScholaWrite the acceptance signal is structurally absent because no model suggestions were ever surfaced to the user. ETA therefore trains on salience and composition signals alone for this benchmark.

A.2 Preprocess Details

This subsection details how raw editing logs become the episode-typed trajectories ETA consumes. We give the per-author statistics, the episode-derivation logic for each benchmark, the LaTeX scaffold-stripping pipeline, and the seven quality filters used to extract salience pairs.

CoAuthor episode derivation. CoAuthor logs writing sessions as Quill Delta event streams of text-insert and text-delete operations interleaved with model-suggestion markers. We segment each session into episodes: every suggestion event starts a new episode, the matching accept-or-reject event closes it, and any free-typed inserts before or between suggestions form manual episodes. Each episode records the document state immediately before the suggestion (before_k), the document state after the suggestion was resolved (after_k), the user’s decision d_k (accepted, rejected, or empty for manual episodes), and the suggestion content p_k . CoAuthor authors are split per genre at a 4:1:1 ratio over six trajectories per genre, yielding $4+1+1 = 6$ trajectories per genre per author and $8+2+2 = 12$ across both genres. The original Mechanical Turk worker IDs are anonymised to $a_1, \dots, a_{10}$ for the paper.

ScholaWrite episode derivation. ScholaWrite logs LaTeX keystrokes paired with full-document snapshots. Successive snapshots are diffed at the paragraph level so that each episode sits inside a single paragraph, and consecutive keystrokes whose timestamps fall within 10 seconds of one another are grouped into a single episode. Because ScholaWrite contains no model suggestions, d_k and p_k are empty for every episode. The seven authors retained are graduate-student contributors to a single Overleaf project whose recorded paragraphs meet a per-author cap of 12 trajectories, with the residual splits anonymised to $a_1, \dots, a_7$ for the paper. Splits are $8+2+2$ per author. Each episode additionally records whether the rejected side of its salience pair is reconstructed against the full document state or against the local paragraph.

Scaffold-stripping filters. Trajectories that contain LaTeX or HTML scaffolding are normalised before pair construction. The stripping pass removes four classes of constructs. The first is sectioning commands (title, section, subsection, subsubsection, paragraph; each with optional starred form). The second is float environments such as figure, table, equation, tabular, tabularx, align, gather, minipage, and subfigure, including their contents between the corresponding begin and end markers. The third is

Table 5: **Hyperparameters across benchmarks.** Production values used to generate the headline numbers in Table 2. Values shared with the canonical CoAuthor recipe carry through to ScholaWrite unless overridden in the right-most column.

Group	Parameter	Value
LoRA	rank (r)	32
	alpha (α)	64
	dropout	0.05
	target modules	$\{q_{\text{proj}}, k_{\text{proj}}, v_{\text{proj}}, o_{\text{proj}}\}$
SFT	scheduler	cosine
	warmup ratio	0.2
	learning rate	3×10^{-5}
	max steps	30 (early-stop at BCE ≈ 1.0)
DPO	scheduler	constant-with-warmup
	warmup ratio	0.25
	learning rate	3×10^{-6}
	max steps	40
ETA-specific	β	0.05
	per-signal weight tuple	rejection 2.0, override 0.5, salience-strict 1.5, salience-fuzzy 1.5, full-lattice 0.5, manual-lattice 2.0

inline annotation commands such as caption, label, anonymous, todo, comment, and includegraphics. The fourth is bare control sequences such as centering and the rule commands of booktabs, plus standalone percent-comment lines. After stripping, pairs falling below 50 characters on either side are dropped. The 50-character minimum is benchmark-agnostic and applies uniformly to ScholaWrite paragraph candidates, lattice-subset composition pairs, and CoAuthor short-continuation guards.

Salience-extractor seven quality filters. The salience signal mines pairs from spans the user actively deleted. Candidate spans pass through seven filters before becoming a preference pair. (1) *Whitespace-or-punctuation-only*: spans with no alphanumeric content are dropped. (2) *Token-count* ≥ 1 after tokenisation. (3) *Alphabetic-word* ≥ 3 characters in at least one word. (4) *Stopword content-word presence*: at least one non-stopword token must be present, ruling out edits that touch only function words. (5) *Typo-similarity*: a span is dropped when its Levenshtein similarity to the surrounding text exceeds 0.7 and the length difference is at most 3 characters, treating the edit as a typo correction rather than a content change. (6) *Scaffolding-phrase regex*: a small allowlist of LaTeX scaffolding phrases is removed even when not caught by the structural strip. (7) *Autocorrect-storm*: spans that occur three or more times in the same trajectory are discarded as autocomplete repetitions rather than authorial choices.

B Experimental Settings

B.1 Hyperparameters and Training Details

All trained methods share the same Mistral-7B-Instruct-v0.2 (Jiang et al., 2023) starting point and the same LoRA (Hu et al., 2022) adapter configuration. The supervised fine-tuning phase uses a cosine learning-rate scheduler with warmup ratio 0.2 and an early-stopping criterion at training BCE loss ≈ 1.0 . The DPO phase uses a constant-with-warmup scheduler at warmup ratio 0.25. All training uses the AdamW optimiser in bfloat16 on a single A100 80GB. Hyperparameters were selected by a four-axis sweep (learning rate, β , max steps, source-weight tuple) on a held-out validation split. Main-results reporting uses GPT-4o for stronger judge calibration. Table 5 lists the production values used for the headline numbers in Table 2.

Per-signal weight tuple. The locked production tuple weights the six pair sources non-uniformly: rejection 2.0, override 0.5, salience-strict 1.5, salience-fuzzy 1.5, full-lattice composition 0.5, manually-anchored composition 2.0. The mean-1 normalisation step (§3.3 of the main text) preserves

592 the effective DPO loss scale, so the tuple is interpreted as relative information density across signals
593 rather than as an absolute scalar dial.

594 B.2 LLM-as-a-judge Settings

595 We evaluate methods head-to-head with an LLM judge, following the protocol of Shaikh et al. (2025).
596 Two methods A and B generate responses for the same prompt, the judge selects whichever response
597 more closely matches an author’s reference text t , and we report the win rate of A over B aggregated
598 across authors and prompts.

599 **Sampling.** The model under evaluation generates 3 responses per prompt at temperature 1.0 with a
600 1024-token generation budget. The judge call itself is deterministic at temperature 0.0.

601 **Eval prompt template.** The exact judge prompt is reproduced verbatim in the box below.

Eval prompt template

```
System message:
You are an impartial evaluator.

User message template:
You are an impartial evaluator. Below is a sample of a human author’s writing and two
options.
### HUMAN AUTHOR’S WRITING:
{demo}
### OUTPUT A:
{text_a}
### OUTPUT B:
{text_b}
### Task
Which option was written by the human author based on similarity to the HUMAN AUTHOR’S
WRITING above? Respond only with a JSON of the following format:
{
  "answer": "<The option most similar to the HUMAN AUTHOR’S WRITING; either A or B>"
}
ALWAYS REMAIN IMPARTIAL WHEN EVALUATING OUTPUTS.
```

602
603 The judge is invoked with the structured-JSON response mode enabled, which constrains the output
604 to a parseable JSON object matching the schema above.

605 B.3 Baselines Details

606 This subsection pins down the prompt template and pair-construction recipes for each baseline. The
607 few-shot and DITTO-step entries reuse infrastructure from Shaikh et al. (2025) where applicable.
608 The zero-shot baseline calls Mistral-7B-Instruct-v0.2 on the bare task prompt with no demonstrations,
609 no system message beyond the model’s default chat template, and no in-context examples, so it does
610 not need a separate template.

611 **Few-shot prompt.** The few-shot baseline prepends the user’s training-set demonstrations as in-
612 context examples. The same template is shared between the GPT-4o and Mistral few-shot baselines
613 so that the prompt construction is identical across them.

Few-shot prompt template

```
Below are a few writing samples.
{examples}
Respond to the following prompt in the same way as the writing samples. Do not generate
output that is GPT-like:
{prompt}
```

614
615 The examples slot is filled by joining each training-set demonstration as a block headed “###
616 EXAMPLE i ” followed by the example’s prompt and output on subsequent lines.

617 **DITTO-step pair construction.** The DITTO-step baseline extends DITTO with a per-episode
618 offline preference pair, evaluated alongside DITTO’s online resampling cache. The offline pair recipe
619 branches on what the trajectory recorded. When the editing UI captured a model proposal p_k for
620 the episode, the rejected side concatenates the in-progress draft state before _{k} with p_k . When the
621 trajectory carries an earlier full-text draft snapshot but no proposal, the rejected side falls back to that
622 snapshot directly. Either way, the offline pair is emitted once at startup and never refreshed, while
623 final-only rows continue to draw their rejected side from DITTO’s online cache, which is refreshed
624 from the live training policy at the start of every training step (resample rate 10). Pseudocode for the
625 offline-pair augmentation:

Algorithm 2 DITTO-step offline-pair augmentation (run at startup). Online pairs are drawn from DITTO’s resampling cache and refreshed every 10 training steps; this pseudocode covers only the offline component.

Require: trajectory $T = [e_1, \dots, e_K]$ with polished final draft y_K and per-episode flag FULL_k indicating whether episode k stores a full-document draft snapshot s_k on its rejected side

```

1:  $\mathcal{P} \leftarrow \emptyset$ 
2: for  $k = 1, \dots, K$  do
3:   if  $d_k$  is accepted then
4:     if  $\text{FULL}_k$  then                                     ▷ snapshot branch
5:       rejected $k$   $\leftarrow s_k$ 
6:     else if  $p_k$  is non-empty then
7:       rejected $k$   $\leftarrow \text{before}_k \parallel \text{“”} \parallel p_k$            ▷ concat branch
8:     else
9:       continue                                           ▷ no offline pair for this episode
10:    end if
11:     $\mathcal{P} \leftarrow \mathcal{P} \cup \{(y_K, \text{rejected}_k)\}$ 
12:  end if
13: end for
14: return  $\mathcal{P}$ 

```

626 The offline-pair set is fixed at startup and never refreshed. On the concat branch, the author’s draft
627 prefix before _{k} is included in the rejected side because that is what the document looked like at the
628 moment of accepting p_k , before the user kept editing past it. At training time the collator splits each
629 batch row-by-row: rows whose accepted episode produced an offline pair use that pair directly, while
630 final-only rows draw a negative from DITTO’s online cache (refreshed every 10 steps from the live
631 training policy). On ScholaWrite, where the editing surface records no model proposals, the offline
632 pair construction routes through the snapshot branch: every accepted episode still emits an offline
633 pair, but the rejected side is an earlier full-document draft rather than a draft-prefix-plus-proposal
634 concatenation. The snapshot variant carries weaker contrastive signal than the concat variant, which
635 is consistent with DITTO-step underperforming DITTO on ScholaWrite. Either way, DITTO-step
636 trains with the same SFT+DPO recipe as DITTO, so any gap to ETA isolates the multi-anchor design
637 from the benefit of using a single recorded edit-history signal as an offline augmentation.

638 **DITTO baseline.** The DITTO baseline performs SFT on the user’s polished demonstrations, then
639 DPO with y_K as the chosen side and online-sampled negatives generated by the model itself during
640 training. The negative pool is refreshed at every training step: the live SFT-trained policy is placed in
641 inference mode, ten responses per prompt are sampled at temperature 1.0 with stochastic decoding,
642 and the resulting samples are cached per-step. Negatives are therefore drawn from the live training
643 policy rather than from a frozen reference snapshot.

644 B.4 Detailed Results

645 This subsection complements Table 2 with two additional views per benchmark: the full pairwise
646 win-rate matrix between all six methods, and the Tukey HSD pairwise comparison table. Both
647 matrices use the same GPT-4o judge, the same author splits, and the same position-swap protocol as
648 the headline numbers.

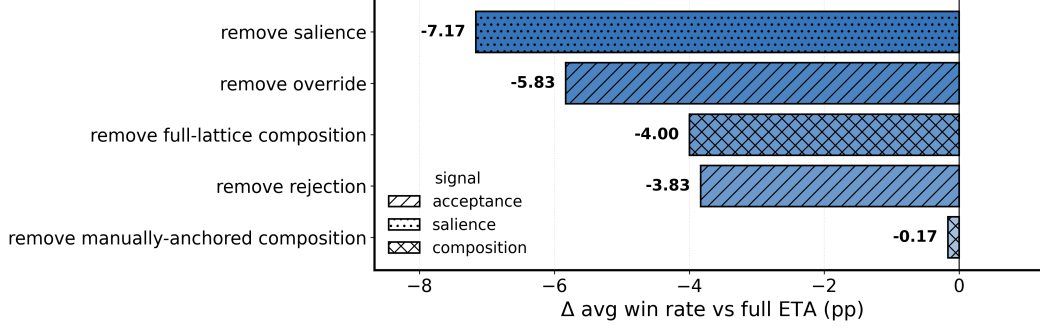


Figure 2: **Source-component ablation, CoAuthor.** Each bar shows the average win-rate drop (in pp) when one preference signal is removed from ETA’s pair construction, computed within a six-method round-robin cohort. Every removal moves win rate downward, and salience contributes the largest amount of unique information. The near-zero drop for manually-anchored composition reflects CoAuthor’s edit profile, where most edits are span-level model acceptances rather than free typing.

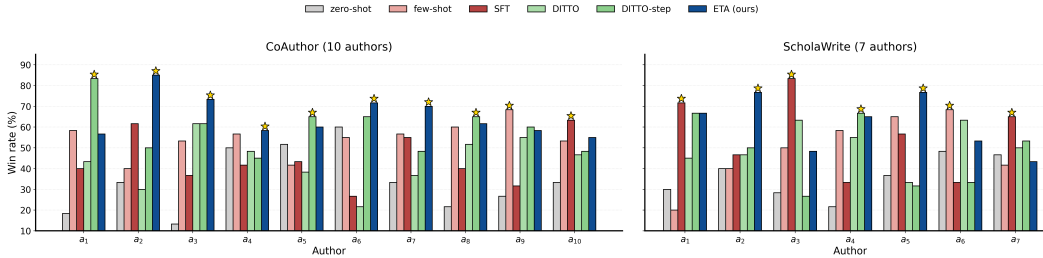


Figure 3: **Per-author win rates, both benchmarks.** Grouped bar chart of head-to-head win rates per author (CoAuthor $a_1, \dots, a_{10}$ on the left, ScholaWrite $a_1, \dots, a_7$ on the right). Star markers (★) above the column-max identify the per-author best method. ETA leads on average on both benchmarks but does not dominate uniformly.

649 **On CoAuthor** ETA’s row dominates the matrix: it wins more than 50% of head-to-heads against
650 every baseline, with the largest margin over zero-shot (78.33%) and the smallest over DITTO-step
651 (54.17%). Tukey HSD confirms ETA’s lead over zero-shot, SFT, and DITTO at $\alpha = 0.05$ but places
652 its lead over DITTO-step (−5.83 pp) inside the non-significance band. The one Tukey-significant
653 pair that does not involve ETA is DITTO vs DITTO-step (−15.83 pp, $p < 0.05$), reflecting the fact
654 that consuming the recorded model proposal as a structured offline negative is itself a strong signal.

655 **On ScholaWrite** ETA’s row again dominates, with the strongest separation against zero-shot
656 (72.62%, the only Tukey-significant pair on this benchmark). Other pairs sit inside the non-
657 significance band, consistent with the smaller per-method sample of $n = 7$ and a per-author standard
658 error around 5 pp. SFT and DITTO are tied in head-to-head win rate (each 50.00%), but SFT pulls
659 ahead in average win rate (55.71% vs 50.95%), suggesting SFT’s gains are concentrated on a few
660 authors rather than spread uniformly.

661 B.5 Per-author data characteristics and the cross-benchmark sign flip

662 We compute eleven per-author metrics from each benchmark’s step-level edit logs: trajectory count,
663 mean episodes per trajectory, proposal density, acceptance rate, manual-edit fraction, deletion volume
664 per trajectory, mean polished-draft length, draft-length stdev, type-token-ratio stdev across drafts,
665 an ETA pair-count proxy, and a DITTO-step pair-count proxy. Joining these per-author metrics
666 with the win-rate matrices yields the figures of merit referenced in §5.1. Figure 4 visualizes the
667 two strongest per-method correlations. On CoAuthor (left panel), DITTO-step’s per-author win rate
668 tracks type-token-ratio stdev across drafts at Spearman $\rho = -0.71$. On ScholaWrite (right panel),

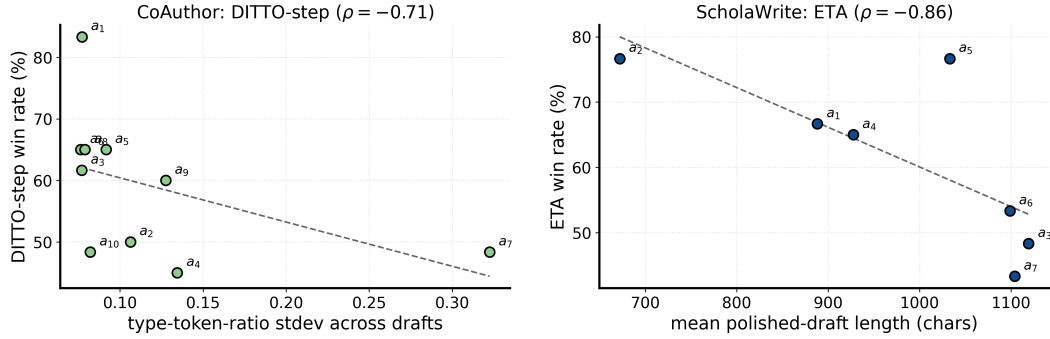


Figure 4: **Per-author win rate correlates with trajectory characteristics.** Left: on CoAuthor ($n = 10$ authors), DITTO-step’s per-author win rate tracks the consistency of an author’s draft-to-draft vocabulary, measured as standard deviation of type-token ratio across that author’s polished drafts (Spearman $\rho = -0.71$). a_1 is the exemplar, with the lowest stylistic variance and the largest DITTO-step win rate in the cohort. Right: on ScholaWrite ($n = 7$ authors), ETA’s per-author win rate correlates negatively with mean polished-draft length ($\rho = -0.86$), the sign-flipped mirror of its CoAuthor pattern. Long drafts carry fewer per-trajectory preference pairs and let SFT’s polished-draft target dominate. Dashed lines are linear-fit guides, not the statistic.

Table 6: **Full pairwise win-rate matrix on CoAuthor (%)**. Each cell reports the head-to-head win rate of the row method over the column method, averaged across the ten authors. The diagonal is 50.00 by construction.

	zero-shot	few-shot	SFT	DITTO	DITTO-step	ETA
zero-shot	50.00	35.83	40.83	40.83	31.67	21.67
few-shot	64.17	50.00	59.17	60.83	46.67	40.83
SFT	59.17	40.83	50.00	50.83	35.83	33.33
DITTO	59.17	39.17	49.17	50.00	35.83	33.33
DITTO-step	68.33	53.33	64.17	64.17	50.00	45.83
ETA	78.33	59.17	66.67	66.67	54.17	50.00

ETA’s per-author win rate tracks mean polished-draft length at $\rho = -0.86$, the sign-flipped mirror of ETA’s CoAuthor pattern at $\rho = +0.59$ on the same metric. The two benchmarks differ structurally rather than statistically. ScholaWrite logs only manual edits, so proposal density is exactly zero on every one of the seven authors, DITTO-step’s proposal-anchored pair construction degenerates, and the polished final draft regains weight as the dominant supervisory signal. On CoAuthor, longer drafts come with richer trajectories (more proposal events, deeper accepted-edit lattice), and ETA’s signal-multiplication advantage scales with draft length. On ScholaWrite, longer drafts dilute the per-pair contrastive signal that trajectory-aware mining depends on, while enriching the polished-draft target that SFT fits.

B.6 Cross-benchmark robustness of multi-anchor mining

Two findings from §5 bear on future work in edit-trajectory alignment. First, methods whose preference-pair construction relies on a single mechanism are brittle in different directions. DITTO uses none of the edit history at all. It trains DPO on the polished final draft against on-policy negatives, so it cannot exploit the per-episode information that distinguishes one user’s editing voice from another’s. DITTO-step augments DITTO with a single recorded edit-history signal, the model proposal on each accepted episode, and is brittle to changes in what the recording surface captures. On ScholaWrite, where no proposals are recorded, the augmentation falls back to a snapshot-based variant whose rejected side is an earlier full-document draft. That fallback carries weaker contrastive signal and underperforms even DITTO. Multi-anchor mining is more robust. ETA’s drop across benchmarks is the smallest in the cohort precisely because the framework adapts to whatever subset of signals a benchmark records. The signals that matter most under ETA’s ablation in §5.1 (salience,

Table 7: **Full pairwise win-rate matrix on ScholaWrite (%)**. Same conventions as Table 6, averaged across the seven ScholaWrite authors.

	zero-shot	few-shot	SFT	DITTO	DITTO-step	ETA
zero-shot	50.00	38.10	33.33	38.10	42.86	27.38
few-shot	61.90	50.00	42.86	48.81	51.19	40.48
SFT	66.67	57.14	50.00	50.00	58.33	46.43
DITTO	61.90	51.19	50.00	50.00	51.19	40.48
DITTO-step	57.14	48.81	41.67	48.81	50.00	38.10
ETA	72.62	59.52	53.57	59.52	61.90	50.00

Table 8: **Tukey HSD pairwise comparisons on CoAuthor**. Mean difference (row – column, in pp of average win rate) and adjusted p -value bucket. ANOVA across the six methods returns $F(5, 54) = 9.57, p < 0.001$.

Method A	Method B	Δ (pp)	p_{adj}
zero-shot	few-shot	−20.17	< 0.01
zero-shot	SFT	−9.83	> 0.05
zero-shot	DITTO	−9.17	> 0.05
zero-shot	DITTO-step	−25.00	< 0.001
zero-shot	ETA	−30.83	< 0.001
few-shot	SFT	+10.33	> 0.05
few-shot	DITTO	+11.00	> 0.05
few-shot	DITTO-step	−4.83	> 0.05
few-shot	ETA	−10.67	> 0.05
SFT	DITTO	+0.67	> 0.05
SFT	DITTO-step	−15.17	> 0.05
SFT	ETA	−21.00	< 0.01
DITTO	DITTO-step	−15.83	< 0.05
DITTO	ETA	−21.67	< 0.01
DITTO-step	ETA	−5.83	> 0.05

690 override, composition) are exactly the ones that ScholaWrite’s keystroke trace still surfaces, even
691 without proposals. Second, when the recording surface is narrow, the polished final draft itself regains
692 weight as a supervisory signal. SFT gains +11.71 pp from CoAuthor to ScholaWrite, the largest gain
693 in the cohort, and closes much of the relative gap to trajectory-aware methods.

Table 9: **Tukey HSD pairwise comparisons on ScholaWrite.** Same conventions as Table 8. ANOVA across the six methods returns $F(5, 36) = 2.38, p > 0.05$, where the smaller author count limits power.

Method A	Method B	Δ (pp)	p_{adj}
zero-shot	few-shot	-13.10	> 0.05
zero-shot	SFT	-19.76	> 0.05
zero-shot	DITTO	-15.00	> 0.05
zero-shot	DITTO-step	-10.95	> 0.05
zero-shot	ETA	-25.48	< 0.05
few-shot	SFT	-6.67	> 0.05
few-shot	DITTO	-1.90	> 0.05
few-shot	DITTO-step	+2.14	> 0.05
few-shot	ETA	-12.38	> 0.05
SFT	DITTO	+4.76	> 0.05
SFT	DITTO-step	+8.81	> 0.05
SFT	ETA	-5.71	> 0.05
DITTO	DITTO-step	+4.05	> 0.05
DITTO	ETA	-10.48	> 0.05
DITTO-step	ETA	-14.52	> 0.05

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: Yes, the claims in the abstract and introduction accurately reflect the paper’s contributions and scope, and are supported by the experimental results (§5).

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: We discuss the limitations of our work in §6.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.

- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [\[Yes\]](#)

Justification: The assumptions and the proof are provided in §3 and §5.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [\[Yes\]](#)

Justification: We disclose all the information needed to reproduce the experimental results in §4.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not include experiments.
- If the paper includes experiments, a [\[No\]](#) answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.

- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We will release data, code, and instructions in the final paper.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The details are specified in §4.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: We reported the mean, standard deviation, and statistical significance in our experiments (§5).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The computer resources are reported.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: This research strictly follows the NeurIPS Code of Ethics.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.

- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: This work provides novel perspectives on learning user preferences from their edit history.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: This work faces no threat of safety.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [\[Yes\]](#)

Justification: The original papers of the datasets used are properly cited. We adhere to the licenses and terms of use as specified by their original creators.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: We will release our codebase along with detailed README files that provide instructions for usage.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [\[N/A\]](#)

Justification: There is no crowdsourcing experiments or research with human subjects in this paper.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

990 Question: Does the paper describe potential risks incurred by study participants, whether
 991 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
 992 approvals (or an equivalent approval/review based on the requirements of your country or
 993 institution) were obtained?

994 Answer: [N/A]

995 Justification: This paper does not include study participants.

996 Guidelines:

- 997 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
 998 with human subjects.
- 999 • Depending on the country in which research is conducted, IRB approval (or equivalent)
 1000 may be required for any human subjects research. If you obtained IRB approval, you
 1001 should clearly state this in the paper.
- 1002 • We recognize that the procedures for this may vary significantly between institutions
 1003 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
 1004 guidelines for their institution.
- 1005 • For initial submissions, do not include any information that would break anonymity (if
 1006 applicable), such as the institution conducting the review.

1007 **16. Declaration of LLM usage**

1008 Question: Does the paper describe the usage of LLMs if it is an important, original, or
 1009 non-standard component of the core methods in this research? Note that if the LLM is used
 1010 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
 1011 scientific rigor, or originality of the research, declaration is not required.

1012 Answer: [N/A]

1013 Justification: LLMs are solely used for editing assistance.

1014 Guidelines:

- 1015 • The answer [N/A] means that the core method development in this research does not
 1016 involve LLMs as any important, original, or non-standard components.
- 1017 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
 1018 be described.